Unit1

INTRODUCTION TO SOFTWARE ENGINEERING

1.1. Introduction To Software Engineering:

- The term software engineering is the product of two words, **software**, and **engineering**.
- The **software** is a collection of integrated programs.
- **Engineering** is the application of scientific and practical knowledge to invent, design, build, maintain, and improve frameworks, processes, etc.
- **Software Engineering** is the process of designing, developing, testing, and maintaining software.
- It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.
- Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.

Why is Software Engineering required?

Software Engineering is required due to the following reasons:

- To manage large software
- For more Scalability
- Cost Management
- To manage the dynamic nature of software
- For better quality Management

Need of Software Engineering:

The necessity of software engineering appears because of a higher rate of progress in user requirements and the environment on which the program is working.

- **Huge Programming:** It is simpler to manufacture a wall than to a house or building, similarly, as the measure of programming become extensive engineering has to step to give it a scientific process.
- **Adaptability:** If the software procedure were not based on scientific and engineering ideas, it would be simpler to re-create new software than to scale an existing one.
- **Cost:** As the hardware industry has demonstrated its skills and huge manufacturing has let down the cost of computer and electronic hardware. But the cost of programming remains high if the proper process is not adapted.
- **Dynamic Nature:** The continually growing and adapting nature of programming hugely depends upon the environment in which the client works. If the quality of the software is continually changing, new upgrades need to be done in the existing one.
- **Quality Management:** Better procedure of software development provides a better and quality software product.

Importance of Software engineering:

1. **Reduces complexity:** Big software is always complicated and challenging to progress. Software engineering has a great solution to reduce the complication of any project. Software engineering divides big problems into various small issues. And then start

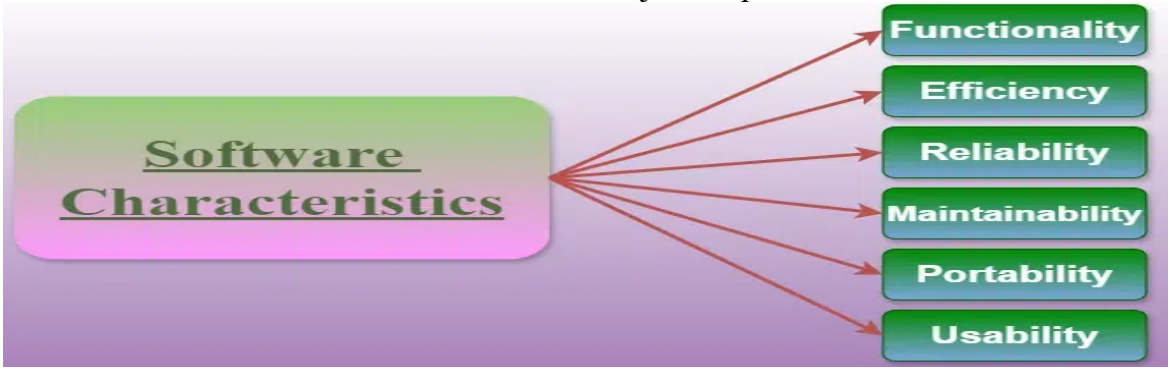
solving each small issue one by one. All these small problems are solved independently to each other.

2. **To minimize software cost:** Software needs a lot of hard work and software engineers are highly paid experts. A lot of manpower is required to develop software with many codes. But in software engineering, programmers project everything and decrease all those things that are not needed. In turn, the cost for software productions becomes less as compared to any software that does not use software engineering method.
3. **To decrease time:** Anything that is not made according to the project always wastes time. And if you are making great software, then you may need to run many codes to get the definitive running code. This is a very time-consuming procedure, and if it is not well handled, then this can take a lot of time. So if you are making your software according to the software engineering method, then it will decrease a lot of time.
4. **Handling big projects:** Big projects are not done in a couple of days, and they need lots of patience, planning, and management. And to invest six and seven months of any company, it requires heaps of planning, direction, testing, and maintenance. No one can say that he has given four months of a company to the task, and the project is still in its first stage. Because the company has provided many resources to the plan and it should be completed. So to handle a big project without any problem, the company has to go for a software engineering method.
5. **Reliable software:** Software should be secure, means if you have delivered the software, then it should work for at least its given time or subscription. And if any bugs come in the software, the company is responsible for solving all these bugs. Because in software engineering, testing and maintenance are given, so there is no worry of its reliability.
6. **Effectiveness:** Effectiveness comes if anything has made according to the standards. Software standards are the big target of companies to make it more effective. So Software becomes more effective in the act with the help of software engineering.

1.2. Nature and Characteristics of Software:

- **Software** is defined as a collection of computer programs, procedures, rules, and data.

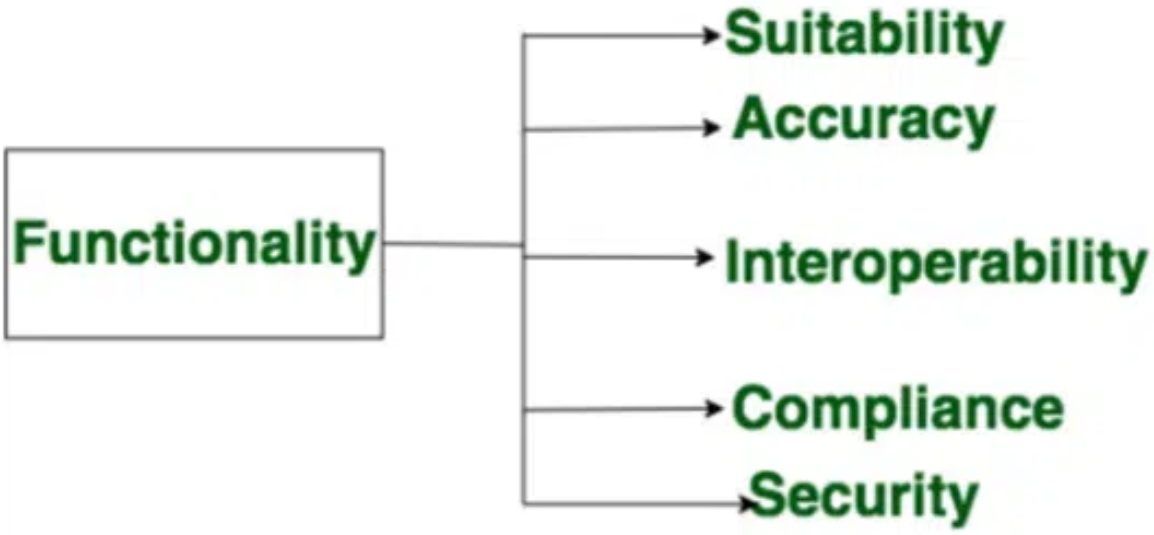
Software Characteristics are classified into six major components.



Functionality:

- It refers to the degree of performance of the software against its intended purpose.
- Functionality refers to the set of features and capabilities that a software program or system provides to its users.
- It is one of the most important characteristics of software, as it determines the usefulness of the software for the intended purpose.

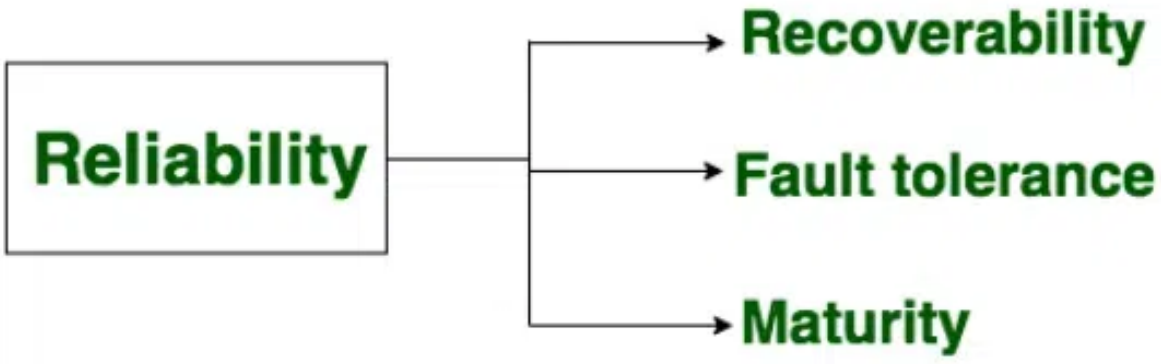
Required functions are:



Reliability:

- A set of attributes that bears on the capability of software to maintain its level of performance under the given condition for a stated period of time.
- Reliability is a characteristic of software that refers to its ability to perform its intended functions correctly and consistently over time.

Required functions are:

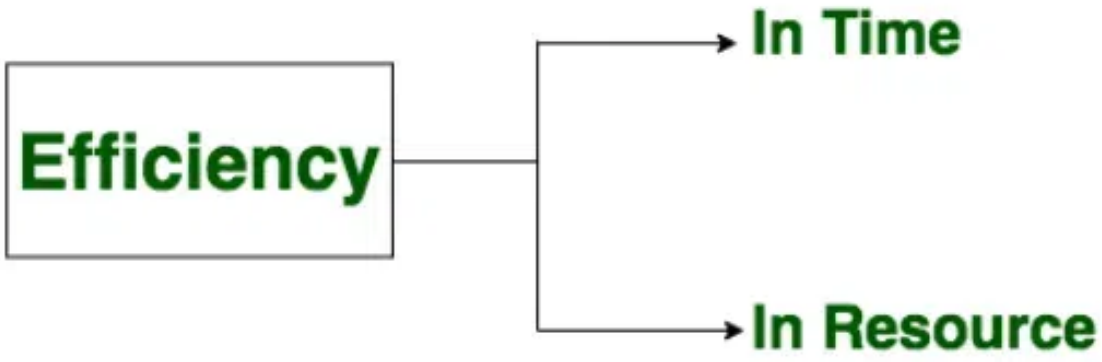


.

Efficiency:

- It refers to the ability of the software to use system resources in the most effective and efficient manner.
- Efficiency is a characteristic of software that refers to its ability to use resources such as memory, processing power, and network bandwidth in an optimal way.

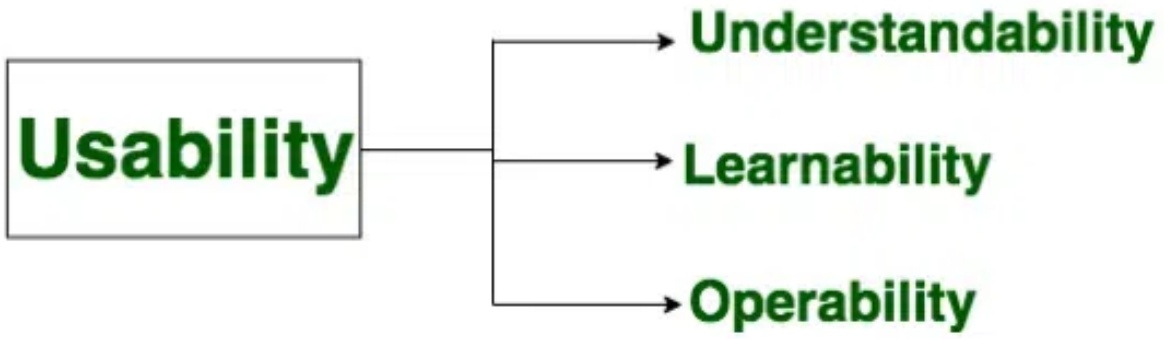
Required functions are:



Usability:

- It refers to the extent to which the software can be used with ease. the amount of effort or time required to learn how to use the software.

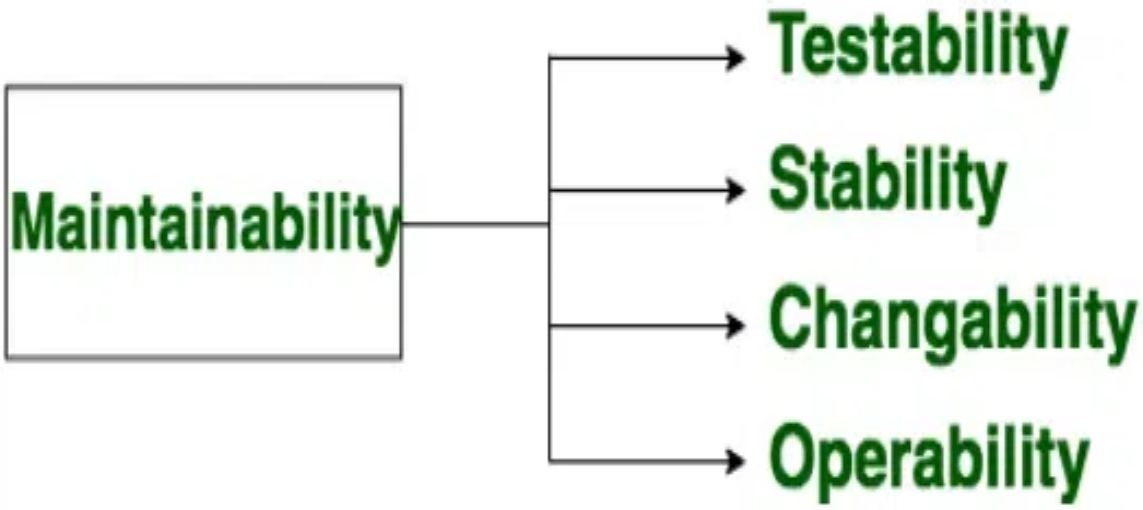
Required functions are:



Maintainability:

- It refers to the ease with which modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.

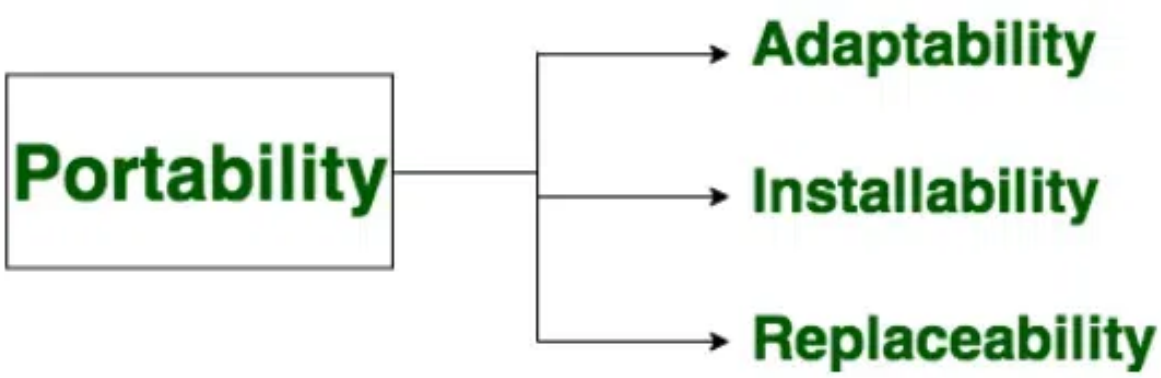
Required functions are:



Portability:

- A set of attributes that bears on the ability of software to be transferred from one environment to another, without minimum changes.

Required functions are:



1.3. Attributes of Good Software:

1. **Functionality:** The software meets the requirements and specifications that it was designed for, and it behaves as expected when it is used in its intended environment.
2. **Usability:** The software is easy to use and understand, and it provides a positive user experience.
3. **Reliability:** The software is free of defects and it performs consistently and accurately under different conditions and scenarios.
4. **Performance:** The software runs efficiently and quickly, and it can handle large amounts of data or traffic.
5. **Security:** The software is protected against unauthorized access and it keeps the data and functions safe from malicious attacks.
6. **Maintainability:** The software is easy to change and update, and it is well-documented, so that it can be understood and modified by other developers.
7. **Reusability:** The software can be reused in other projects or applications, and it is designed in a way that promotes code reuse.
8. **Scalability:** The software can handle an increasing workload and it can be easily extended to meet the changing requirements.
9. **Testability:** The software is designed in a way that makes it easy to test and validate, and it has a comprehensive test coverage.

1.4. Software Engineering versus System Engineering:

Aspect	Software Engineering	System Engineering
Focus	Primarily focuses on software development and processes.	Focuses on designing and managing complex systems.
Scope	Narrower scope	Broader scope
Objectives	Develops software applications or systems to meet specific requirements.	Integrates multiple components (hardware, software, processes) to achieve system goals.
Emphasis	Emphasizes software design, coding, testing, and maintenance.	Emphasizes system architecture, integration, and optimization.
Lifecycle	Follows SDLC methodologies.	Involves system lifecycle management from conception to retirement.
Disciplines	Incorporates disciplines like requirements engineering, software design, and quality assurance.	Integrates disciplines such as system architecture, system integration, and system testing.
Skills	Requires strong programming skills, knowledge of algorithms, and software design patterns.	Requires systems thinking, problem-solving skills, and understanding of interdisciplinary concepts.
Examples	Examples: application development, web development, and mobile app development.	Example: aerospace systems, transportation systems, and healthcare systems.

1.5. Software Engineering Ethics and Professional Practice:

Goal:

- To Establish a code of conduct for professional software engineers to make software engineering a beneficial and respected profession.
- This is a Joint Effort by lEEE-Computer Society and Association of Computing MachineryACM

Software engineers are those who contribute either by direct participation or by

- teaching,
- analysing,
- Specification generating,
- designing,
- developing,
- certifying,
- Maintaining and
- testing

of software system

Roles of Engineers

- ✓ "Professional Software Engineers" include

Practitioners
Educators
Managers
Supervisors
Policy makers

Trainees and Students of the Profession

Software Engineering Ethics:

1. **PUBLIC**

- Software engineers shall act consistently with the public interest
- Accept full responsibility for their own work.
- Moderate the interests of the software engineer, the employer, the client and the users with the public good
- Approve software only if they believe that it is safe, meets specifications, passes appropriate tests
- Be fair and avoid deception in all statements, particularly public ones
- Consider issues of physical disabilities and allocation of resources
- Be encouraged to volunteer professional skills to good cause

2. **CLIENT AND EMPLOYER**

- Software engineers shall act in a manner that is in the best interests of their client and employer, consistent with the public interest
- Provide service in their areas of competence
- Not knowingly use software that is obtained or retained either illegally or unethically.
- Use the property of a client or employer only in ways properly authorized
- Identify, document, collect evidence and report to the client or the employer promptly if, a project is likely to fail or to violate intellectual property.

3. **PRODUCT**

- Software engineers shall ensure that their products and related modifications meet the highest professional standards possible
- Strive for high quality and acceptable cost
- Ensure proper and achievable goals and objectives for any project
- Ensure that they are qualified for any project they work on
- Ensure that an appropriate method is used for any project
- Work to follow professional standards
- Strive to fully understand the specifications for software
- Ensure adequate testing, debugging, documentation and review of software
- Treat all forms of software maintenance with the same professionalism as new development

4. **JUDGMENT**

- Software engineers shall maintain integrity and independence in their professional judgment
- Temper all technical judgments by the need to support and maintain human values.
- Only endorse documents if prepared under supervision
- Maintain professional objectivity with respect to any software
- Not engage in deceptive financial practices such as double billing, or other improper financial practices.
- Disclose to all concerned parties those conflicts of interest that cannot reasonably be avoided or escaped

5. **MANAGEMENT**

- Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance
- Ensure good management for any project on which they work
- Ensure that software engineers are informed of standards before being held to them.
- Ensure realistic quantitative estimates of cost, scheduling, personnel, quality and outcomes on any project
- Provide for due process in hearing charges of violation of an employer's policy or of this Code.
- Not ask a software engineer to do anything inconsistent with this Code.
- Not punish anyone for expressing ethical concerns about a project

6. **PROFESSION**

- Software engineers shall advance the integrity and reputation of the profession consistent with the public interest
- Help develop an organizational environment favourable to acting ethically
- Promote public knowledge of software engineering
- Support, as members of a profession, other software engineers striving to follow this Code.
- Not promote their own interest at the expense of the profession, client or employer.
- Take responsibility for detecting, correcting, and reporting errors in software
- Report significant violations of this Code to appropriate authorities.

7. **COLLEAGUES**
- Software engineers shall be fair to and supportive of their colleagues
- Encourage colleagues to adhere to this Code
- Assist colleagues in professional development
- Credit fully the work of others and refrain from taking undue credit
- Assist colleagues in being fully aware of current standard work practices
- Not unfairly intervene in the career of any colleague

8. **SELF**
- Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession
- Further their knowledge of recent developments
- Improve their ability to create safe, reliable, and useful quality software
- Improve their ability to produce accurate, informative, and well-written documentation
- Improve their knowledge of relevant standards
- Not influence others to undertake any action that involves a breach of this Code.

Professional Practices:

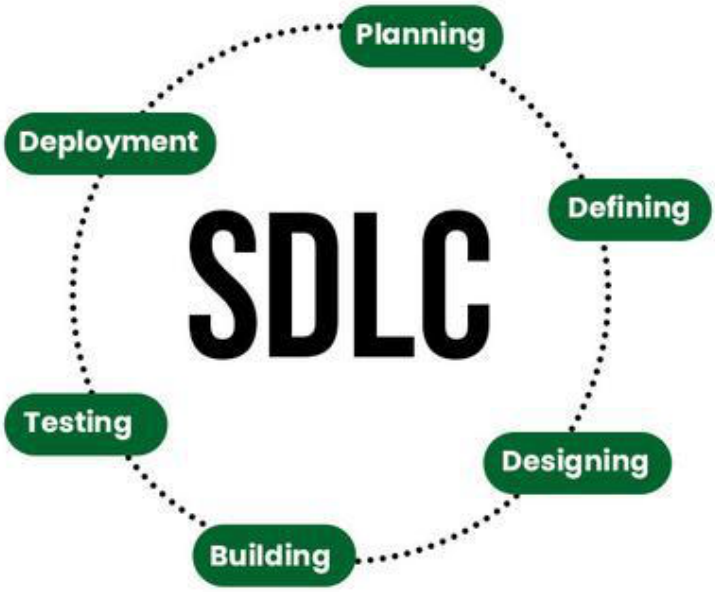
- **Continuous Learning:** Staying up-to-date with the latest technologies and best practices is crucial for maintaining professional competence.
- **Clear Communication:** Effective communication with clients, colleagues, and users is essential for understanding requirements, collaborating effectively, and addressing issues.
- **Accountability:** Engineers should take responsibility for their work, acknowledging mistakes and striving for improvement.
- **Teamwork:** Collaboration with other professionals is vital for successful software development, fostering a positive and productive work environment.
- **Adherence to Standards:** Following established coding standards, guidelines, and regulations ensures consistency and quality in software development.

Ethical Challenges in Software Engineering:

- **Privacy and Security:** Protecting user data and ensuring the security of software systems are major ethical concerns.
- **Bias and Discrimination:** Developers must be mindful of potential biases in algorithms and ensure fairness in software design.
- **Environmental Impact:** The energy consumption and carbon footprint of software systems should be considered.
- **Intellectual Property:** Respecting copyright laws and avoiding plagiarism are essential.
- **Social Responsibility:** Software engineers should consider the broader societal impact of their work and avoid creating harmful or discriminatory software.

1.6. Software Development Lifecycle (SDLC):

- Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software.
- SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step.
- SDLC is a process followed for software building within a software organization.
- SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software.
- The life cycle defines a method for improving the quality of software and the all-around development process



.

Stages of the Software Development Life Cycle

SDLC is a collection of these six stages, and the stages of SDLC are as follows:

Stage-1: Planning and Requirement Analysis

- Planning is a crucial step in everything, just as in software Development.
- In this same stage, requirement analysis is also performed by the developers of the organization.
- This is attained from customer inputs, and sales department/market surveys.
- The information from this analysis forms the building blocks of a basic project.
- The quality of the project is a result of planning.
- Thus, in this stage, the basic project is designed with all the available information.

Stage-2: Defining Requirements

- In this stage, all the requirements for the target software are specified.
- These requirements get approval from customers, market analysts, and stakeholders.
- This is fulfilled by utilizing SRS (Software Requirement Specification).
- This is a sort of document that specifies all those things that need to be defined and created during the entire project cycle.

Stage-3: Designing Architecture

- SRS is a reference for software designers to come up with the best architecture for the software.
- Hence, with the requirements defined in SRS, multiple designs for the product architecture are present in the Design Document Specification (DDS).
- This DDS is assessed by market analysts and stakeholders.
- After evaluating all the possible factors, the most practical and logical design is chosen for development.

Stage-4: Developing Product

- At this stage, the fundamental development of the product starts.
- For this, developers use a specific programming code as per the design in the DDS.
- Hence, it is important for the coders to follow the protocols set by the association.
- Conventional programming tools like compilers, interpreters, debuggers, etc. are also put into use at this stage.
- Some popular languages like C/C++, Python, Java, etc. are put into use as per the software regulations.

Stage-5: Product Testing and Integration

- After the development of the product, testing of the software is necessary to ensure its smooth execution.
- Although, minimal testing is conducted at every stage of SDLC.
- Therefore, at this stage, all the probable flaws are tracked, fixed, and retested.
- This ensures that the product confronts the quality requirements of SRS.

Stage-6: Deployment and Maintenance of Products

- After detailed testing, the conclusive product is released in phases as per the organization's strategy. Then it is tested in a real industrial environment.
- It is important to ensure its smooth performance.
- If it performs well, the organization sends out the product as a whole.
- After retrieving beneficial feedback, the company releases it as it is or with auxiliary improvements to make it further helpful for the customers.